EventBooking – Functional Specification

I want to significantly upgrade the existing EventBooking project.

The current system is mainly focused on booking event halls. I want to turn it into a complete **event planning and booking platform** where a customer can build an entire event in one place.

The main concept is:

> **Everything needed to plan an event – in one place.**

The customer should be able to choose a venue, catering, table design, bridal chair when relevant, additional services, communicate with managers, receive notifications and see promotions.

IMPORTANT – READ BEFORE CODING

The project already exists:

- EventBooking-Server
- EventBooking-Client

Do NOT rebuild the project from scratch.

First analyze the existing architecture and code.

Preserve the existing:

- Architecture
- Entities
- Controllers
- Services
- DTOs
- Authentication
- Authorization
- Database structure where possible
- Existing booking logic
- Existing HallSlot/Booking concurrency mechanism

Extend the existing system instead of creating a parallel architecture.

Before making major changes, briefly explain:

1. What currently exists
2. What needs to change
3. Which new entities/features are needed
4. Why each change is necessary

PHASE 1 – FUNCTIONALITY FIRST

For this phase, focus on:

- Backend functionality
- Database
- Business logic
- API
- React pages/components required for functionality

Do NOT redesign the React UI yet.

Do not spend time on advanced CSS, animations, colors or visual redesign.

We will do a separate UI/UX redesign after the functionality is complete.

---

1. Event Type

Add an event type to an event/booking.

Possible types:

- Wedding
- Bar Mitzvah
- Bat Mitzvah
- Corporate Event
- Birthday
- Private Event
- Other

The event type should determine which services are relevant.

For example, Wedding can have:

- Bridal Chair
- Table Design
- Catering
- Photography
- DJ
- Flowers

The design should be extensible for future event types.

---

2. Upgrade ExtraService

The existing project already contains `ExtraService`.

Reuse it instead of creating unnecessary duplicate entities.

Add a service type/category, for example:

- Catering
- TableDesign
- BridalChair
- Photography
- DJ
- Flowers
- Lighting
- Other

An ExtraService should support properties such as:

- Id
- Name
- Description
- Price
- ServiceType
- ImageUrl
- IsActive
- ManagerId/OwnerId according to the existing architecture

The goal is to make services extensible.

---

3. Bridal Chair

For Wedding events, allow the customer to select a Bridal Chair service.

The customer should be able to see:

- Whether the service is available
- Style
- Description
- Price
- Image

This can be implemented using `ExtraService` with `ServiceType = BridalChair`.

Do not create a separate entity unless there is a strong architectural reason.

---

4. Table Design

Add Table Design as a service.

It can contain:

- Style
- Colors
- Tablecloth
- Napkins
- Centerpieces
- Flowers
- Tableware
- Custom design
- Price

This can also use `ExtraService` with `ServiceType = TableDesign`.

---

5. Catering

Catering should have more business logic than a normal extra service.

Create a suitable `CateringMenu` entity if appropriate.

Possible properties:

- Id
- Name
- Description
- PricePerGuest
- IsVegetarian
- IsVegan
- IncludesDrinks
- IsActive
- ManagerId

The price should depend on the number of guests.

Example:

300 guests × 180 ₪ per guest = 54,000 ₪

The final calculation must happen on the **server side**.

Never trust a total price sent by the React client.

---

6. Event Builder

Create a flow where the customer can build an event step by step:

1. Event type
2. Date
3. Number of guests
4. Venue
5. Catering
6. Table design
7. Bridal Chair if relevant
8. Additional services
9. Price summary
10. Booking confirmation

The user should be able to review all selected services before confirming.

---

7. Price Calculation

Create a proper server-side price calculation.

Example:

Venue: 8,000 ₪

Catering: 300 × 180 = 54,000 ₪

Table Design: 2,500 ₪

Bridal Chair: 1,000 ₪

DJ: 3,500 ₪

Total: 69,000 ₪

The server must calculate the total using prices stored in the database.

The client must NOT be trusted to provide the final price.

---

8. Messaging System

Add a messaging system between customers and managers.

A customer should be able to send a message to a venue/service manager.

Example:

Customer: "Hello, I would like to check availability for this date."

The manager should see the message and be able to reply.

Create a suitable `Message` entity containing:

- Id
- SenderId
- ReceiverId
- BookingId/EventId where appropriate
- Content
- CreatedAt
- IsRead

Managers should be able to see unread messages.

Customers should be able to see replies.

---

9. Notifications

Add a notification system.

Possible notifications:

- New message
- Booking approved
- Booking rejected
- Booking cancelled
- New promotion
- Booking update

Users should be able to see unread notification count.

---

10. Promotions

Add a `Promotion` entity.

Possible properties:

- Id
- Title
- Description
- DiscountPercentage or DiscountAmount
- StartDate

- EndDate
- ImageUrl
- IsActive
- ManagerId

Managers should be able to create and manage promotions.

Example:

"15% off table design this month"

Only active promotions within their valid date range should be displayed.

---

11. Promotion Popup / Modal

Add support for displaying an active promotion or important announcement in the React application.

Example:

🎉 New Promotion

20% off event table design

The user should be able to close the modal.

Keep this functionality simple for now.

We will redesign the popup visually in the UI/UX phase.

---

12. Customer Dashboard

Create/upgrade the customer dashboard.

The customer should be able to see:

- My Events
- My Bookings
- Selected Services
- Price Estimates
- Messages
- Notifications
- Promotions

---

13. Manager Dashboard

Create/upgrade the manager dashboard.

Managers should be able to see:

- New bookings
- Existing bookings
- Customers
- Messages
- Services they offer
- Promotions
- Availability
- Basic activity statistics

A manager must only be able to manage resources that belong to them.

---

14. Search and Filtering

Improve venue search.

Customers should be able to filter by:

- Event type
- Number of guests
- Price
- Date
- Area/location
- Services
- Availability

The filtering should be implemented properly on the backend where appropriate.

---

15. Reviews

Add a review/rating system.

A customer should be able to review a venue/service after a relevant booking or completed event.

Example:

- Rating: 1–5
- ReviewText

Important:

A user should not be able to create fake reviews without having a relevant booking.

---

16. Authorization

Keep the existing roles:

- Customer
- Manager
- Admin

Rules:

Customer:

- Can manage their own events/bookings
- Can send messages
- Can view relevant services/promotions

Manager:

- Can manage their own venues/services
- Can manage their own promotions
- Can view relevant customer messages/bookings

Admin:

- Can manage the entire system

Authorization must be enforced on the **server side**, not only in React.

---

17. React Changes

Only make the React changes required to support the new functionality.

Add/update where necessary:

- Pages
- Components
- API services
- React Query queries/mutations
- Routes
- Forms
- Data models/types

However:

Do NOT perform the visual redesign yet.

Keep the UI functional and simple for now.

---

18. Database Design

Before adding new entities, inspect the existing database model.

Avoid duplicate entities and relationships.

Reuse existing structures whenever reasonable.

Potential entities:

User Event Venue Hall HallSlot Booking ExtraService BookingExtraService CateringMenu Message Notification Promotion Review

Adapt this list to the existing architecture rather than blindly creating all of them.

---

19. API

Add appropriate API endpoints for the new functionality.

Potential areas:

- Events
- Bookings
- Services
- Catering
- Messages
- Notifications
- Promotions
- Reviews

Follow the existing architecture.

Use:

- DTOs
- Validation
- Appropriate HTTP status codes
- Authorization
- Error handling
- Existing Service/Repository patterns if already used

Do not introduce unnecessary architectural patterns.

---

20. IMPORTANT – Preserve Existing Concurrency Logic

The existing project contains logic related to booking `HallSlot` and preventing conflicting bookings.

Do NOT remove or weaken this mechanism.

If the project already uses transactions, concurrency/version checking or HTTP 409 responses for booking conflicts, preserve that behavior.

The system must continue to correctly handle two users attempting to book the same slot.

---

21. Testing

After implementing the changes:

- Build the backend
- Run the backend
- Run the React client
- Verify the database
- Test the new API endpoints
- Test authorization
- Test price calculation
- Test booking conflicts
- Test messaging
- Test promotions
- Test that existing functionality still works

Create/update migrations where necessary.

---

22. Educational Requirement

This is a software engineering study project.

The code must be understandable and explainable.

Prefer:

- Clear naming
- Simple architecture
- Separation of responsibilities
- Meaningful DTOs
- Reasonable validation
- Minimal but useful comments

Do not add complexity just to make the project look advanced.

I need to be able to explain the implementation to a teacher.

---

23. Final Report

When finished, provide a clear summary.

Backend

Explain:

- New entities
- Modified entities
- New controllers
- Modified controllers
- New services
- Modified services
- New DTOs
- New API endpoints
- Database relationships
- Migrations

Frontend

Explain:

- New pages
- New components
- New API services
- New React Query queries/mutations
- New routes

Business Logic

For each major feature explain:

- What problem it solves
- How the data flows
- Which backend components participate
- Which frontend components participate

Keep the explanation simple enough for a software engineering student to understand.

FINAL PRODUCT VISION

The final application should evolve from:

"An event hall booking system"

into:

"A complete event planning and booking platform."

The customer should be able to build an entire event:

Venue

+ Catering

+ Table Design

+ Bridal Chair when relevant

+ Additional Services

+ Price Calculation

+ Booking

+ Messaging

+ Notifications

+ Promotions

All in one system.

Again:

Do the functionality first.

We will handle the complete visual redesign separately after this phase is stable.